Università Politecnica delle Marche

Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione



Progetto Ricostruzione NerfStudio

COMPUTER GRAPHICS

Cingolani Filippo
Pettinaro Alessadro
Sebastianelli Nicola

Anno accademico: 2025–2026

Indice

1 Introduzione al progetto **3**

2 Fondamenti teorici **4**

2.1 Neural Radiance Fields (NeRF) 4

2.2 3D Gaussian Splatting (3DGS) 5

2.3 NerfStudio 7

3 Metodologia: la pipeline di ricostruzione **8**

3.1 Pipeline 8

3.1.1 I dati di input 9

3.1.2 La conversione delle immagini 10

3.1.3 Preparazione del dataset per NerfStudio 10

3.1.4 Addestramento dei modelli 12

3.1.5 Esportazione degli output 12

3.1.6 Valutazione dei modelli 13

Elenco delle figure

1 Schema della pipeline di ricostruzione del progetto. 8
2 Forma dataset 9
3 file jscon camera 9

1 Introduzione al progetto

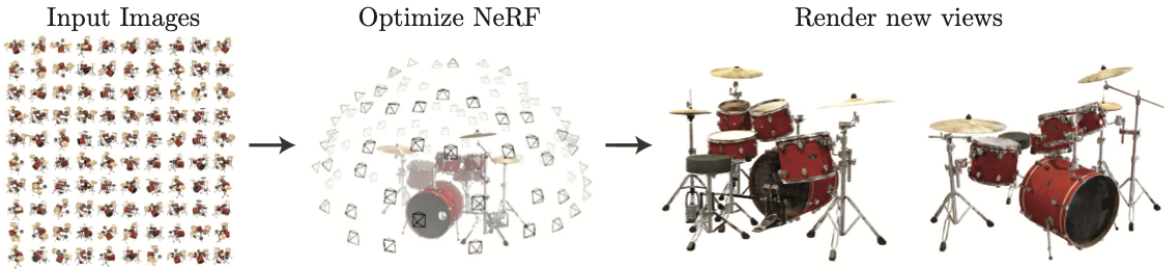
La digitalizzazione 3D di oggetti reali è oggi uno dei campi più attivi della computer vision. Rispetto alle tecniche tradizionali fotogrammetria classica e scansione laser, costose e spesso poco fedeli nel restituire i dettagli superficiali gli approcci neurali come NeRF (Neural Radiance Fields) e 3D Gaussian Splatting (3DGS) permettono di ottenere modelli tridimensionali di alta qualità a partire da semplici sequenze di immagini. Questo li rende particolarmente adatti alla digitalizzazione del patrimonio artigianale, dove è fondamentale preservare non solo la forma, ma anche la complessità materica e i dettagli più minuti. Il progetto, sviluppato in collaborazione con Bottega Intreccio, si concentra su manufatti intrecciati: oggetti che pongono sfide specifiche ai sistemi di ricostruzione, per via di elementi sottili (i singoli fili), cavità e occlusioni, pattern ripetitivi che ostacolano la stima delle pose, e riflessioni superficiali variabili. L'obiettivo è costruire una pipeline completa di ricostruzione 3D su dati reali già raccolti, sfruttando i dati a disposizione (immagini multi-vista, pose delle camere e mappe di profondità). Il lavoro non si limita a un singolo metodo, ma mette a confronto diversi modelli di ricostruzione disponibili in NerfStudio (basati su NeRF e su 3D Gaussian Splatting) con l'obiettivo di individuare, attraverso un'analisi critica dei risultati, quale approccio offra le prestazioni migliori sui diversi oggetti analizzati.

2 Fondamenti teorici

Prima di descrivere la pipeline sviluppata, è utile introdurre le tecnologie su cui essa si fonda. Questo capitolo fornisce il background necessario a comprendere il lavoro svolto, presentando i due paradigmi di ricostruzione 3D adottati e il framework che ne ha reso possibile l'implementazione e il confronto. Si parte dai Neural Radiance Fields (NeRF), per poi passare al 3D Gaussian Splatting (3DGS). Infine viene presentato NerfStudio, il framework modulare che integra entrambi i metodi all'interno di un'unica infrastruttura e che costituisce la piattaforma operativa dell'intero progetto.

2.1 Neural Radiance Fields (NeRF)

Il primo dei due paradigmi di ricostruzione adottati in questo lavoro è quello dei Neural Radiance Fields (NeRF), introdotto da Mildenhall et al. nel 2020. L'obiettivo del metodo è la sintesi di nuove viste: a partire da un insieme di immagini di un oggetto acquisite da angolazioni diverse, con le pose della camera note, si vogliono generare viste fotorealistiche da punti di osservazione arbitrari, mai effettivamente fotografati. La scena viene così ricostruita in tre dimensioni a partire da sole immagini bidimensionali.



L'elemento distintivo di NeRF è il modo in cui la scena viene rappresentata. Anziché ricorrere a una rappresentazione esplicita, come una mesh o una griglia di voxel, la scena è codificata in modo implicito e continuo nei pesi di una rete neurale completamente connessa (un percettrone multistrato, o MLP). La rete approssima una funzione del tipo $(\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$, dove $\mathbf{x} = (x, y, z)$ è la posizione di un punto nello spazio, $\mathbf{d}$ è la direzione di osservazione, $\mathbf{c} = (r, g, b)$ è il colore emesso e σ è la densità volumetrica.

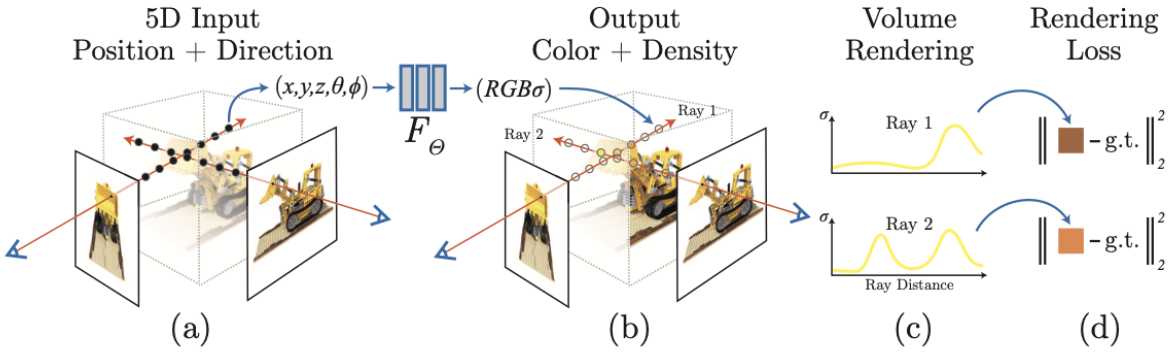
La generazione di un'immagine avviene tramite *volume rendering*, secondo una procedura articolata in quattro passi

1. *(a)* Per ogni pixel si lancia un raggio nella scena e lungo di esso si campionano alcune coordinate 5D, date dalla posizione del punto e dalla direzione di osservazione.

2. *(b)* Queste coordinate vengono passate in input all'MLP, che restituisce per ciascun punto un colore e una densità volumetrica.

3. *(c)* I valori così ottenuti vengono combinati tramite le tecniche di volume rendering in una somma pesata, che fornisce il colore finale del pixel: il peso di ciascun punto dipende dalla sua densità e da quanto è visibile, ovvero da quanta materia è già stata incontrata lungo il raggio prima di esso, cosicché i punti occlusi contribuiscono meno.

4. *(d)* Poiché l'intero procedimento è differenziabile, la rete può essere addestrata minimizzando l'errore tra le immagini reali e quelle renderizzate dal modello. Imponendo questa coerenza simultaneamente su molte viste, la rete è spinta a costruire una rappresentazione consistente della scena, assegnando densità e colori corretti solo dove è effettivamente presente il contenuto reale.



Il principale vantaggio di NeRF risiede nella combinazione di alta qualità e compattezza: codificando il volume nei parametri di una rete, si evita il costo di memoria proibitivo delle griglie di voxel ad alta risoluzione. Il limite principale è invece il costo computazionale, sia in addestramento sia in rendering, dovuto al numero elevato di valutazioni della rete richieste per ogni raggio.

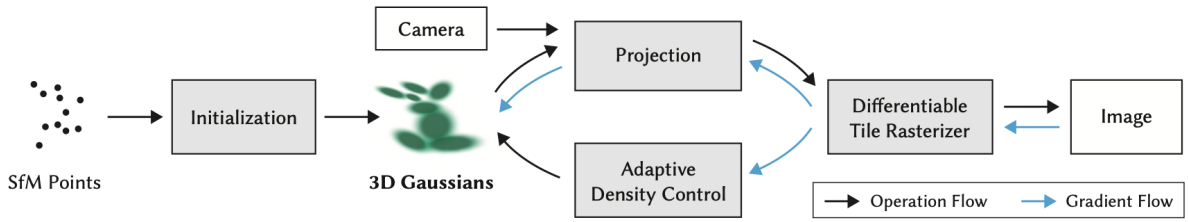
2.2 3D Gaussian Splatting (3DGS)

Il secondo paradigma di ricostruzione adottato in questo lavoro è il 3D Gaussian Splatting (3DGS), proposto da Kerbl et al. nel 2023. L'obiettivo è il medesimo di NeRF sintetizzare nuove viste a partire da un insieme di immagini ma con una finalità aggiuntiva esplicita: ottenere un rendering *in tempo reale* ad alta risoluzione, mantenendo al contempo una qualità visiva pari allo stato dell'arte e tempi di addestramento competitivi. È proprio sulla velocità che i metodi basati su NeRF mostravano il loro limite più grande, dovuto al costoso campionamento stocastico lungo i raggi.

L'idea chiave è abbandonare la rappresentazione implicita e continua di NeRF in favore di una rappresentazione *esplicita*: la scena viene descritta come un insieme di gaussiane tridimensionali, ciascuna assimilabile a un piccolo ellissoide "sfocato" nello spazio. Ogni gaussiana è definita da una posizione (il suo centro), una matrice di covarianza che ne determina forma, dimensione e orientamento, un valore di opacità e un colore. La covarianza è *anisotropa*, cioè la gaussiana può allungarsi liberamente in una direzione: questo permette di rappresentare in modo compatto anche strutture sottili e dettagli fini, allineando le gaussiane alla geometria della superficie. Il colore, inoltre, è codificato tramite armoniche sferiche, una rappresentazione che consente di riprodurre gli effetti dipendenti dalla direzione di vista, ossia i riflessi.

La pipeline parte dallo stesso input di NeRF, cioè le immagini con le pose della camera calibrate tramite Structure-from-Motion (SfM). Un vantaggio pratico è che il processo SfM produce "gratuitamente" una nuvola di punti sparsa, che viene usata per *inizializzare* le gaussiane: ogni punto diventa una gaussiana, e da questo insieme iniziale si parte per l'ottimizzazione. I parametri di ogni gaussiana (posizione, covarianza, opacità e colore) vengono ottimizzati in modo iterativo, confrontando le immagini renderizzate con quelle reali e correggendoli per minimizzare l'errore.

Un elemento distintivo è il *controllo adattivo della densità*: durante l'ottimizzazione l'algoritmo non si limita a regolare i parametri, ma aggiunge e rimuove gaussiane dove serve. Nelle regioni in cui la geometria è poco rappresentata le gaussiane vengono duplicate o suddivise, mentre quelle praticamente trasparenti (con opacità sotto una soglia) vengono eliminate. Questo permette di passare da una nuvola iniziale sparsa a una rappresentazione densa e accurata, mantenendola comunque compatta.



La vera differenza rispetto a NeRF sta però nel rendering. Mentre NeRF ricostruisce il colore di un pixel campionando molti punti lungo un raggio (*volume rendering*), il 3DGS adotta un approccio inverso basato sulla *rasterizzazione*: le gaussiane 3D vengono proiettate sul piano immagine della camera (diventando "splat" 2D), ordinate per profondità e fuse pixel per pixel tramite alpha-blending, dalla più vicina alla più lontana. È interessante notare che il modello di formazione dell'immagine è in sostanza lo stesso di NeRF ma l'algoritmo concreto è profondamente diverso. Poiché la rasterizzazione è

un'operazione estremamente efficiente sulle GPU, ed è qui realizzata con un rasterizzatore "a tile" (che suddivide lo schermo in riquadri e ordina le gaussiane una sola volta per l'intera immagine), il rendering può avvenire in tempo reale. Anche questo procedimento è interamente differenziabile, condizione necessaria per poter addestrare il modello. In sintesi, il modello 3DGS presenta vantaggi e svantaggi ben definiti rispetto agli approcci basati su NeRF.

Vantaggi:

- qualità visiva pari o superiore allo stato dell'arte precedente (Mip-NeRF360);
- tempi di addestramento nell'ordine di pochi minuti, anziché di ore o giorni;
- rendering in tempo reale a oltre 100 fps in alta risoluzione, contro i tempi nell'ordine dei secondi per fotogramma dei metodi NeRF di pari qualità.

Svantaggi:

- occupazione di memoria sensibilmente maggiore rispetto alle soluzioni basate su NeRF, dovuta alla natura esplicita della rappresentazione;
- comparsa di artefatti (gaussiane allungate o "a chiazze") nelle regioni poco osservate o nelle viste molto diverse da quelle di addestramento.

2.3 NerfStudio

Il framework su cui si basa l'intera pipeline è NerfStudio, una piattaforma open source sviluppata all'Università di Berkeley e presentata a SIGGRAPH 2023, nata per semplificare lo sviluppo e l'uso pratico delle tecniche di neural rendering. La sua caratteristica principale è la **modularità**: la pipeline di ricostruzione è scomposta in componenti intercambiabili (gestione dei dati, modello, rendering ed esportazione), così che metodi diversi possano condividere la stessa infrastruttura. Questo aspetto è particolarmente rilevante per il presente lavoro, poiché consente di confrontare più approcci sugli stessi dati in modo equo, variando soltanto il modello e mantenendo invariato il resto della catena.

NerfStudio integra sotto un'unica infrastruttura sia i metodi basati su NeRF tra cui Nerfacto, che combina diverse tecniche per bilanciare qualità e velocità sia il 3D Gaussian Splatting, tramite l'implementazione Splatfacto basata sulla libreria *gsplat* accelerata su GPU. I due paradigmi descritti in precedenza sono dunque entrambi disponibili all'interno dello stesso ambiente. Il framework offre inoltre un flusso di lavoro completo, dalla preparazione dei dati all'addestramento fino all'esportazione dei risultati, e include un visualizzatore web in tempo reale che permette di osservare la ricostruzione nel corso dell'addestramento.

3 Metodologia: la pipeline di ricostruzione

Questo capitolo descrive la metodologia operativa del progetto: il percorso che trasforma i dati grezzi dell'acquisizione in un modello tridimensionale ricostruito e visualizzabile in NerfStudio. La pipeline è presentata in forma generale, indipendente dallo specifico manufatto, così da poter essere applicata in modo uniforme a oggetti diversi.

3.1 Pipeline

L'obiettivo della pipeline è trasformare i dati di un'acquisizione fotografica in un modello tridimensionale dell'oggetto, ricostruito con tecniche di neural rendering e pronto per essere visualizzato, confrontato e riutilizzato. Si parte da un insieme di immagini riprese da molti punti di vista, accompagnate da informazioni di supporto come le pose delle camere e le mappe di profondità, oltre a un modello 3D di riferimento acquisito separatamente; si arriva a una scena ricostruita da cui è possibile generare nuove viste, video e geometria esportabile.Poiché il progetto prevede di trattare più oggetti e di mettere a confronto tecniche diverse, la pipeline è concepita come un procedimento unico e ripetibile: gli stessi passaggi si applicano a ogni oggetto e a ogni metodo, così che i risultati restino confrontabili in modo coerente. È per questa ragione che la descrizione è volutamente generale definisce il flusso una sola volta mentre le scelte specifiche e i risultati dei singoli casi trovano spazio nei capitoli sperimentali.

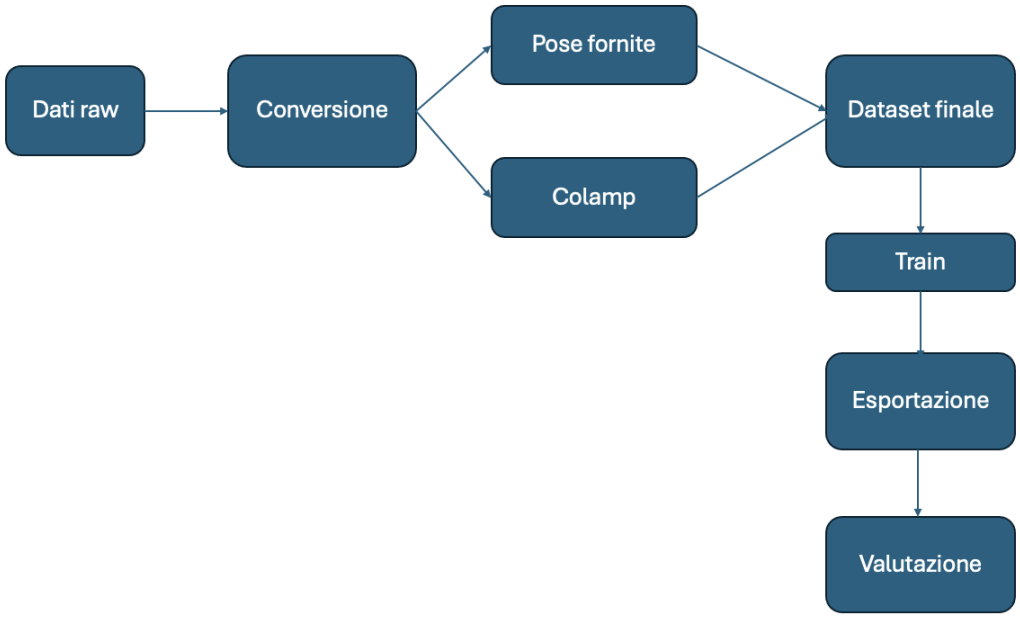


Figura 1: Schema della pipeline di ricostruzione del progetto.

3.1.1 I dati di input

Per ogni oggetto, il materiale di partenza è organizzato in una struttura di cartelle, ciascuna dedicata a un tipo di informazione. Affiancato a queste c'è inoltre un modello tridimensionale di riferimento, acquisito separatamente. La forma complessiva del dataset è la seguente :

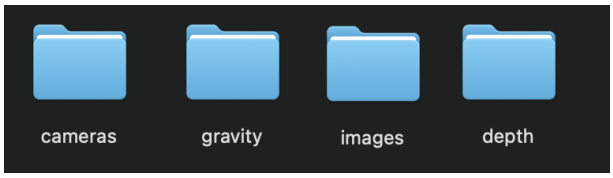


Figura 2: Forma dataset

1. La cartella **images** contiene le fotografie dell'oggetto, riprese da molti punti di vista attorno ad esso. Sono salvate in formato HEIF.

2. La cartella **cameras** contiene un file JSON per ciascuna immagine, con i parametri della camera relativi a quello specifico scatto: la sua posa, cioè dove si trovava e come era orientata nello spazio, e i suoi parametri ottici interni. Un file ha la forma seguente:


```
{
 "t_00" : 0.47910815,
 "t_02" : -0.5930064,
 "fy" : -0.70238096,
 "t_01" : 0.6471467,
 "t_23" : 0,
 "t_12" : -0.7504636,
 "t_13" : 0,
 "height" : 0,
 "fx" : -0.70238096,
 "t_22" : -0.291801,
 "cy" : -0.85119045,
 "t_10" : -0.037602574,
 "cx" : -0.85119045,
 "t_11" : -0.6598413,
 "t_03" : 0,
 "t_21" : 0.3818518,
 "blur_score" : 222.64137,
 "width" : 0,
 "gravity" : {
   "y" : -0.6598412394523621,
   "z" : -0.750463604927063,
   "x" : -0.037602558732032776
 },
 "t_20" : -0.87695014
}
```


Figura 3: file jscon camera

In breve, i valori da t_00 a t_23 formano la matrice di posa della camera (rotazione e traslazione), fx, fy, cx, cy sono i parametri ottici interni, gravity ripete la direzione della gravità per quello scatto e blur_score ne misura la nitidezza

3. La cartella **depth** contiene una mappa di profondità in formato PNG per ciascuna immagine. In una mappa di profondità il valore di ogni pixel non rappresenta un colore, ma la distanza stimata tra la camera e la superficie inquadrata in quel punto.

4. La cartella **gravity** riporta la direzione della gravità misurata dai sensori inerziali del dispositivo al momento di ogni scatto.

3.1.2 La conversione delle immagini

Il primo passo concreto del preprocessing consiste nel convertire le immagini dal formato HEIF a un formato di immagine standard come il PNG. Questo passaggio è necessario perché, come anticipato, l'HEIF è il formato di acquisizione dei dispositivi iOS e non è leggibile nativamente da NerfStudio né dalla maggior parte degli strumenti della pipeline: lasciare le immagini in HEIF significherebbe, semplicemente, non poterle dare in ingresso al sistema di ricostruzione.

3.1.3 Preparazione del dataset per NerfStudio

Una volta convertite le immagini, queste da sole non sono sufficienti per addestrare un modello. Le tecniche di ricostruzione basate su viste multiple, come NeRF e 3D Gaussian Splatting, non imparano la forma di un oggetto dalle sole fotografie: hanno bisogno di sapere, per ciascuna immagine, da dove è stata scattata e con quale ottica. Sono proprio queste informazioni (la posa della camera e i suoi parametri interni) a permettere al modello di collegare tra loro le diverse viste e di ricostruire una rappresentazione tridimensionale coerente.

NerfStudio si aspetta che queste informazioni siano raccolte in un preciso formato di ingresso: un unico file, chiamato `transforms.json`, che accompagna la cartella delle immagini e descrive l'intera geometria di acquisizione della scena. Lo scopo di questa fase è quindi costruire tale file, ottenendo alla fine un dataset completo e pronto per l'addestramento.

La costruzione del `transforms.json` può seguire due strade alternative, che si differenziano per il modo in cui si ottengono le pose delle camere. La prima utilizza le pose già fornite insieme ai dati, convertendole nel formato richiesto; la seconda le stima da zero a partire dalle sole immagini, tramite COLMAP. Le due strade conducono allo stesso risultato, ma per vie diverse e con vantaggi e limiti differenti, come vedremo nelle sezioni che seguono.

1. La prima strada non usa le pose fornite, ma le ricostruisce dalle sole immagini tramite COLMAP, lo strumento di Structure-from-Motion integrato in NerfStudio. Il principio è ricavare insieme la struttura 3D della scena e la posizione delle camere osservando come gli stessi punti dell'oggetto compaiono in fotografie diverse: COLMAP individua punti caratteristici nelle immagini, li mette in corrispondenza tra una vista e l'altra e da queste corrispondenze stima le pose e i parametri ottici di ogni camera. In pratica il comando `ns-process-data` di NerfStudio riceve la cartella delle immagini, esegue COLMAP e produce direttamente il `transforms.json` nel formato e nella convenzione corretti, senza alcuna conversione manuale.

2. La seconda strada sfrutta il fatto che il dataset contiene già, nella cartella `cameras/`, la posa e i parametri ottici di ogni immagine. Non occorre quindi stimarli: il lavoro consiste nel tradurre queste informazioni dal formato dell'app di cattura a quello di NerfStudio, raccogliendole nell'unico file `transforms.json`. La traduzione si svolge nei passi seguenti.

   - **Recupero delle dimensioni delle immagini.** I file delle camere riportano i campi `width` e `height` impostati a 0, quindi la larghezza e l'altezza reali non sono leggibili da lì: vanno ricavate direttamente dalle immagini convertite, ed è qui che torna utile aver preservato la risoluzione nel passo precedente. Queste dimensioni servono sia a compilare i campi `w` e `h` del file, sia per il passo successivo.
   - **Denormalizzazione dei parametri ottici.** Nei dati grezzi `fx`, `fy`, `cx`, `cy` sono normalizzati e compresi tra -1 e 1, mentre NerfStudio li vuole espressi in pixel; vanno quindi riportati in scala usando la larghezza e l'altezza appena recuperate.
   - **Composizione della matrice di posa.** I dodici valori da `t_00` a `t_23` vengono riorganizzati nella matrice 3×4 (le prime tre colonne per l'orientamento, l'ultima per la traslazione) e completati con la riga $[0, 0, 0, 1]$ per ottenere la matrice 4×4 attesa da NerfStudio.
   - **Conversione del sistema di coordinate.** La convenzione degli assi usata dall'app di cattura non coincide con quella di NerfStudio, e le pose vanno convertite di conseguenza. È il punto più delicato, perché un errore qui non blocca l'esecuzione, ma produce ricostruzioni in cui le camere puntano nel verso sbagliato e il modello non converge; per questo la correttezza si verifica osservando l'orientamento delle camere nel visualizzatore di NerfStudio.

Completati questi passi, i parametri ottici denormalizzati e l'elenco di tutte le pose convertite confluiscono nel `transforms.json`, che insieme alla cartella delle immagini costituisce il dataset pronto per l'addestramento.

3.1.4 Addestramento dei modelli

Ottenuto il dataset nel formato richiesto, si procede con l'addestramento vero e proprio. Questa fase non consiste in un singolo addestramento, ma nella scelta e nella prova di diversi modelli e configurazioni, da confrontare tra loro: mantenendo fisso il dataset e facendo variare il metodo o i suoi iperparametri, si isola l'effetto di ciascuna scelta sulla qualità della ricostruzione.

NerfStudio mette a disposizione metodi appartenenti a due famiglie. La prima è quella basata su NeRF, con metodi come `nerfacto` e la variante `depth-nerfacto`, che sfrutta anche le mappe di profondità del dataset. La seconda è quella basata su 3D Gaussian Splatting, con il metodo `splatfacto`.

Le configurazioni messe alla prova variano lungo alcune direzioni principali:

- **Numero di iterazioni.** Si fa variare la durata dell'addestramento per osservare come la qualità della ricostruzione evolve e a partire da quante iterazioni il modello converge senza ulteriori miglioramenti apprezzabili.
- **Uso della profondità.** Con i metodi NeRF che la supportano (`depth-nerfacto`) si confrontano gli addestramenti con e senza supervisione di profondità, intervenendo anche sul peso della relativa funzione di perdita (*depth loss*) per misurare quanto l'informazione geometrica del LiDAR aiuti la ricostruzione.
- **Metodo di ricostruzione.** Si confrontano i risultati dei metodi NeRF con quelli dello splatting (`splatfacto`), variandone gli iperparametri specifici e il numero di iterazioni, per valutare come le due famiglie si comportino sullo stesso oggetto.

Al termine di ogni addestramento il modello ottenuto viene salvato, pronto per l'esportazione degli output e per la successiva valutazione.

3.1.5 Esportazione degli output

A partire da ciascun modello addestrato, NerfStudio consente di esportare diversi tipi di output, che costituiscono il materiale su cui si baserà la valutazione dei singoli modelli. L'esportazione viene quindi ripetuta per ogni modello e configurazione provati, in modo da poterli confrontare a parità di condizioni.

Gli output prodotti sono di tre tipi principali.

- **Render di nuove viste.** Sono immagini dell'oggetto generate dal modello da punti di vista scelti, comprese angolazioni non presenti tra le fotografie originali (*novel views*). Permettono di valutare la resa visiva della ricostruzione e la sua coerenza al variare del punto di osservazione.

- **Video dimostrativi.** Definendo una traiettoria di camera attorno all'oggetto, si esporta un video che mostra la ricostruzione in movimento. Oltre a essere un materiale di presentazione richiesto in fase di consegna, il video evidenzia eventuali incoerenze o artefatti che emergono solo osservando la scena da una sequenza continua di viste.

- **Geometria della scena.** Si esporta la forma tridimensionale ricostruita, sotto forma di mesh o di nuvola di punti per i metodi NeRF, oppure dell'insieme delle gaussiane per lo splatting. Questo output ha un ruolo particolare, perché è la geometria a poter essere confrontata con il modello di riferimento acquisito con lo scanner, rendendo possibile una valutazione non solo visiva ma anche quantitativa dell'accuratezza della forma.

L'insieme di questi materiali (render, video e geometria) raccolto per ciascun modello addestrato costituisce la base su cui si svolge la fase di valutazione, descritta nella sezione seguente.

3.1.6 Valutazione dei modelli

Ogni modello addestrato viene valutato in due fasi. La prima è un confronto visivo: si esaminano i render delle nuove viste e il video, osservando la fedeltà all'oggetto reale, la resa degli intrecci e dei dettagli e la presenza di eventuali artefatti, in particolare nelle cavità e nelle zone di sovrapposizione. La seconda è una valutazione quantitativa tramite metriche, distinte in due gruppi a seconda che richiedano o meno un'immagine di riferimento.

Metriche full-reference Confrontano il render con l'immagine reale corrispondente, presa dallo stesso punto di vista; richiedono quindi che le viste di test abbiano una foto di riferimento.

- **PSNR** (*Peak Signal-to-Noise Ratio*). Si basa sull'errore quadratico medio (MSE) tra immagine di riferimento I e immagine ricostruita K, entrambe di dimensione $m \times n$:

$$MSE = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [I(i,j) - K(i,j)]^2, \qquad PSNR = 10 \cdot \log_{10}\left(\frac{MAX^2}{MSE}\right)$$

dove MAX è il massimo valore di un pixel. Si esprime in decibel; più alto è meglio. È una misura puramente pixel-per-pixel: non riflette pienamente la percezione umana, e un piccolo disallineamento o una lieve sfocatura possono abbassarla anche quando l'immagine appare buona.

- **SSIM** (*Structural Similarity Index Measure*). Nasce come alternativa percettiva al PSNR e misura la similarità strutturale combinando luminanza, contrasto e struttura su finestre locali:

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

  con C_1, C_2 piccole costanti di stabilità. Il valore è compreso tra -1 e 1 (in pratica tra 0 e 1); più alto è meglio. È sensibile a bordi e trama, quindi rilevante per un oggetto intrecciato.

- **LPIPS** (*Learned Perceptual Image Patch Similarity*). Invece di confrontare i pixel, passa le due immagini attraverso una rete profonda pre-addestrata e misura la distanza tra le feature interne, pesata su giudizi umani. È una distanza percettiva: 0 indica immagini identiche, più basso è meglio.

- **depth_MSE**. È l'MSE applicato alle mappe di profondità anziché ai colori: confronta la profondità renderizzata dal modello con quella di riferimento del sensore. Più basso è meglio. È l'unica metrica puramente geometrica del gruppo (indica se le superfici sono collocate correttamente nello spazio) ed esiste solo per i modelli che producono profondità.

Metriche no-reference Valutano il render da solo, senza confronto con un riferimento; giudicano la qualità visiva in assoluto ma non la fedeltà alla scena reale.

- **BRISQUE** (*Blind/Referenceless Image Spatial Quality Evaluator*). Si basa sulle *Natural Scene Statistics*: le immagini naturali presentano regolarità statistiche che le distorsioni alterano in modo misurabile. Dai coefficienti di luminanza normalizzati localmente estrae feature da cui un modello predice la qualità. Più basso è meglio.

- **CLIP-IQA**. Sfrutta il modello vision-language CLIP per stimare la qualità senza addestramento specifico, tramite coppie di prompt antonimi (ad esempio "Good photo." contro "Bad photo."). Il punteggio è la probabilità relativa che l'immagine sia più vicina al prompt positivo, ed è compreso tra 0 e 1; più alto è meglio.